



UNIT 5

SOFTWARE TESTING

INTRODUCTION TO SOFTWARE TESTING

- Software Testing is a method to check whether the actual software product matches expected requirements and to ensure that software product is defect free.
- Software Testing is important because if there are any bugs or errors in the software, it can be identified early and can be solved before delivery of the software product.
- To perform testing, we create test cases and test data. Test case is a collection of actions which is applied to our software product to check the specific features and functionality of it. Test data is the input provided to the modules which are present in our software product.

TEST CASE

- Documentation plays a critical role in testing.
- There are different levels of documentation, like:
 1. **Test Script:** A line-by-line description of all the actions and data needed to perform a test.
 2. **Test Case:** Describes a specific idea that is to be tested, without detailing the exact steps to be taken or data to be used.
 3. **Test Scenario:** It is a simple description of an objective a user might face when testing.

TEST CASE FORMAT

- **Test Case Name:** A test case should have a name or title that is self-explanatory.
- **Test Case Description:** The description should tell the tester what they're going to test in brief.
- **Pre-Conditions:** Any assumptions that apply to the test and any preconditions that must be met prior to the test being executed should be listed here.
- **Test Case Steps:** The test steps should include the necessary data and information on how to execute the test. The steps should be clear and brief, without leaving out essential facts.

TEST CASE FORMAT

- **Test Data:** It's important to select a data set that gives sufficient coverage. Select a data set that specifies not only the positive scenarios but negative ones as well.
- **Expected Result:** The expected results tell the tester what they should experience as a result of the test steps.
- **Actual Result:** They specifies how the application actually behaved while test cases were being executed.
- **Comments:** Any other useful information such as screenshots that tester want's to specify can be included here.

EXAMPLE OF TEST CASE

Title: Login Page – Authenticate user on Hotmail.com

Description: A user should be able to log in at hotmail.com.

Precondition: The user must have an email address and password that is previously registered.

Assumption: The browser supports hotmail.com

Test Steps:

1. Navigate to hotmail.com
2. Enter the email address of the registered user in the 'email' field.
3. Enter the password of the registered user
4. Click the 'Next' button.
5. Click 'Log in'

Expected Result: The Hotmail inbox of the user should load.

EXAMPLE OF TEST CASE

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
TU01	Check Customer Login with valid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	UserId = guru99 Password = pass99	User should Login into an application	As Expected	Pass
TU02	Check Customer Login with invalid Data	1. Go to site http://demo.guru99.com 2. Enter UserId 3. Enter Password 4. Click Submit	UserId = guru99 Password = glass99	User should not Login into an application	As Expected	Pass

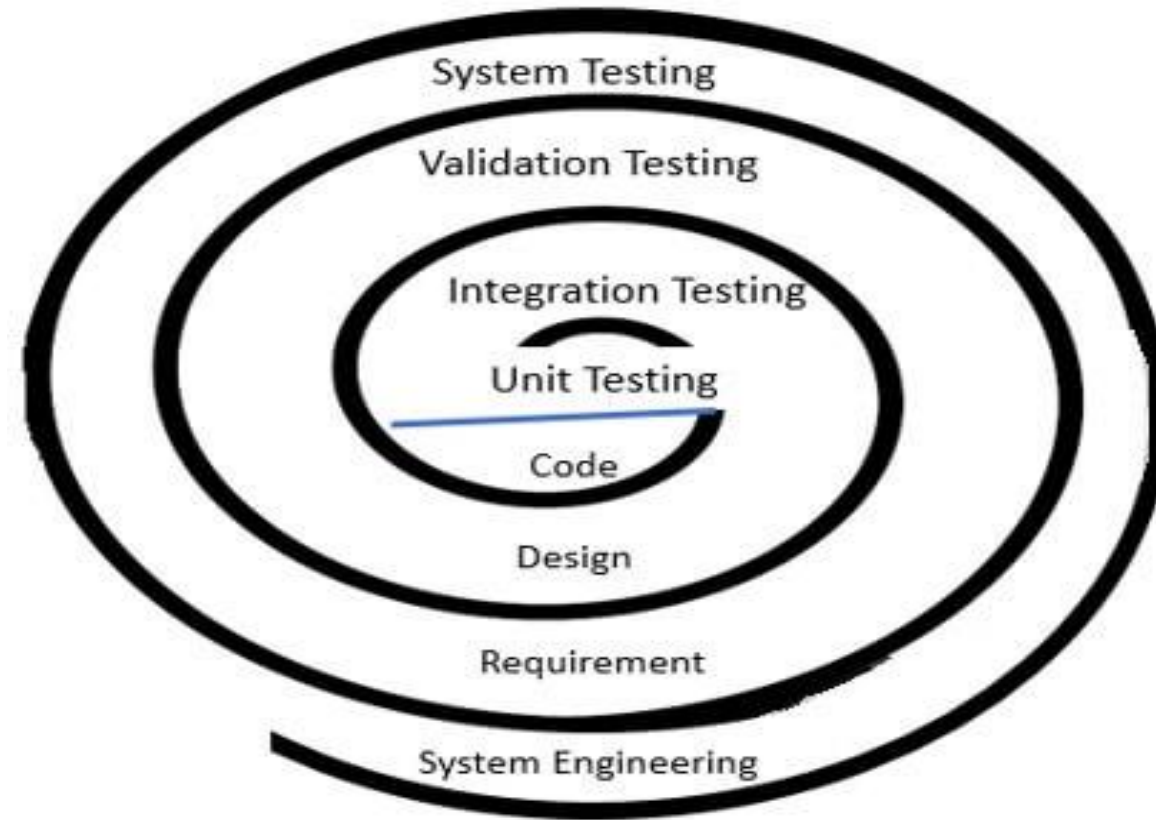
SOFTWARE TESTING - Objectives

1. Uncover as many as errors as possible in a given software product.
2. Demonstrate a product to match its requirement specifications.
3. Validate the quality of a software using the minimum cost and efforts.
4. Generate high-quality test cases, perform effective tests, and issue correct and helpful problem reports.

SOFTWARE TESTING - Principles

1. All the test should meet the customer requirements
2. To make our software, testing should be performed by a third party
3. As we need the optimal amount of testing based on the risk assessment of the application.
4. All the test to be conducted should be planned before implementing it
5. It follows the Pareto rule(80/20 rule) which states that 80% of errors come from 20% of program components.
6. Start testing with small parts and extend it to large parts.

TESTING STRATEGIES



Software Testing Strategy

TESTING STRATEGIES

- For testing, initially, the core part of the software is tested, each unit of the software is tested separately to ensure that as a unit this component functions properly.
- Then the testing proceeds outward the spiral along the streamlines.
- Moving outwards the spiral in the clockwise direction gradually widens the scope of testing and in each turn, testing integrates the components
- Also it ensure that all the elements of the software are functioning properly and the desired performance is achieved.

VALIDATION

- Validation is the process of checking whether the software product is up to the mark or a product has high level requirements.
- It is the process of checking the validation of product i.e. it checks what we are developing is the right product.
- It is validation of actual and expected product.
- Validation is the **Dynamic Testing**.

VALIDATION

Activities involved in validation:

- 1.Black box testing
- 2.White box testing
- 3.Unit testing
- 4.Integration testing

COMPARISON OF VERIFICATION VALIDATION

VERIFICATION	VALIDATION
1. It includes checking documents, design, codes and programs.	1. It includes testing and validating the actual product.
2. Verification is the static testing.	2. Validation is the dynamic testing.
3. Methods used in verification are reviews, walkthroughs, inspections and desk-checking.	3. Methods used in validation are Black Box Testing, White Box Testing and non-functional testing.
4. It checks whether the software conforms to specifications or not.	4. It checks whether the software meets the requirements and expectations of a customer or not.

COMPARISON OF VERIFICATION VALIDATION

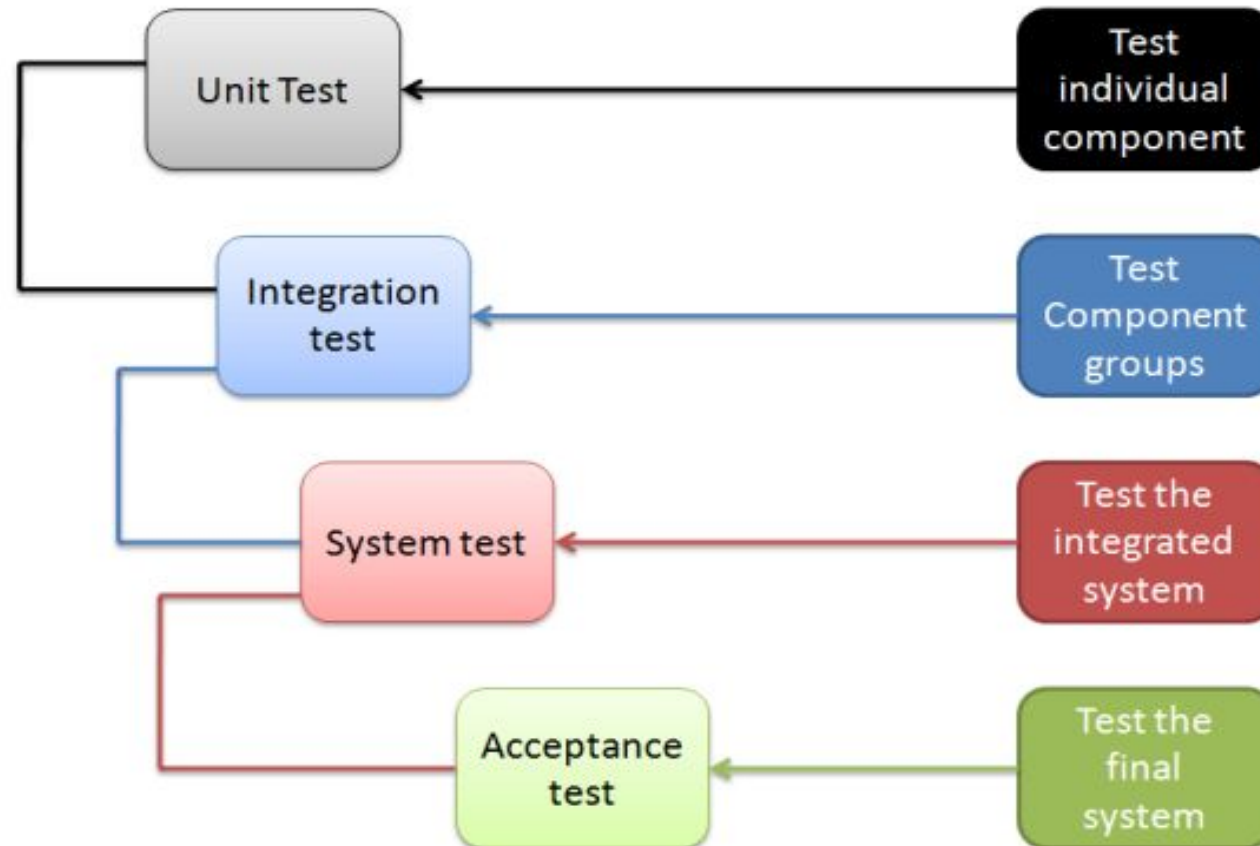
VERIFICATION	VALIDATION
5. It comes before validation.	5. It comes after verification.
6. It does <i>not</i> include the execution of the code.	6. It includes the execution of the code.
7. It can find the bugs in the early stage of the development.	7. It can only find the bugs that could not be found by the verification process.
8. Verification is done by QA team to ensure that the software is as per the specifications in the SRS document.	8. Validation is executed on software code with the help of testing team.

MANUAL VS AUTOMATED TESTING

Testing can either be done manually or using an automated testing tool:

- 1. Manual** - This testing is performed without taking help of automated testing tools. The software tester prepares test cases for different sections and levels of the code, executes the tests and reports the result to the manager. Manual testing is time and resource consuming.
- 2. Automated** - This testing is a testing procedure done with the help of automated testing tools. The limitations with manual testing can be overcome using automated test tools.

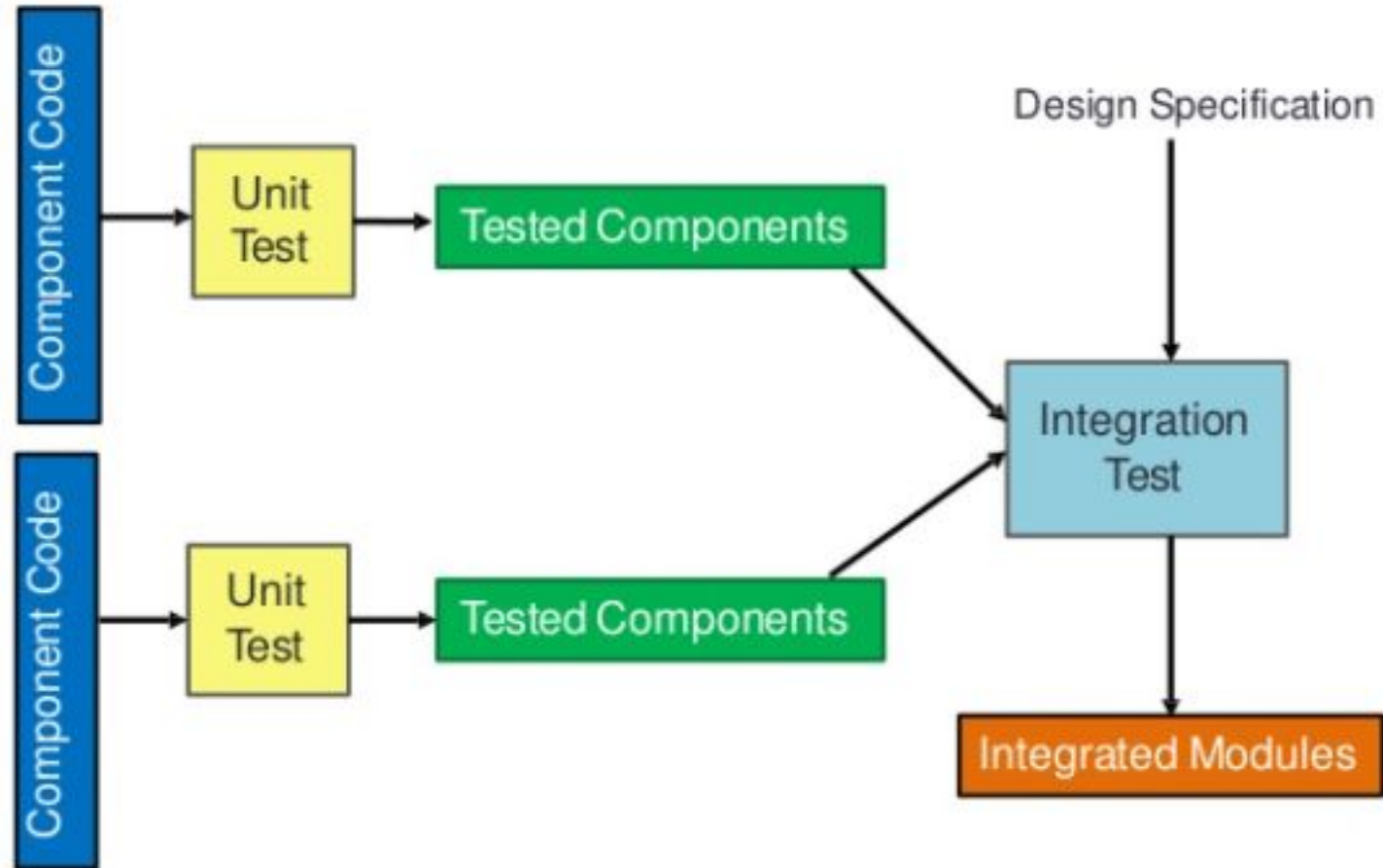
LEVELS OF TESTING



UNIT TESTING

- Unit Testing, also known as Component Testing, is a level of software testing where individual units / components of a software are tested.
- The purpose is to validate that each unit of the software.
- It checks that the components are fulfilling functionalities or not.
- Unit Testing is usually performed by using the White Box Testing method
- This kind of testing is performed by developers.

UNIT TESTING



UNIT TESTING - Objectives

1. To isolate a section of code.
2. To verify the correctness of code.
3. To test every function and procedure.
4. To fix bug early in development cycle and to save costs.
5. To help the developers to understand the code base and enable them to make changes quickly.
6. To help for code reuse.

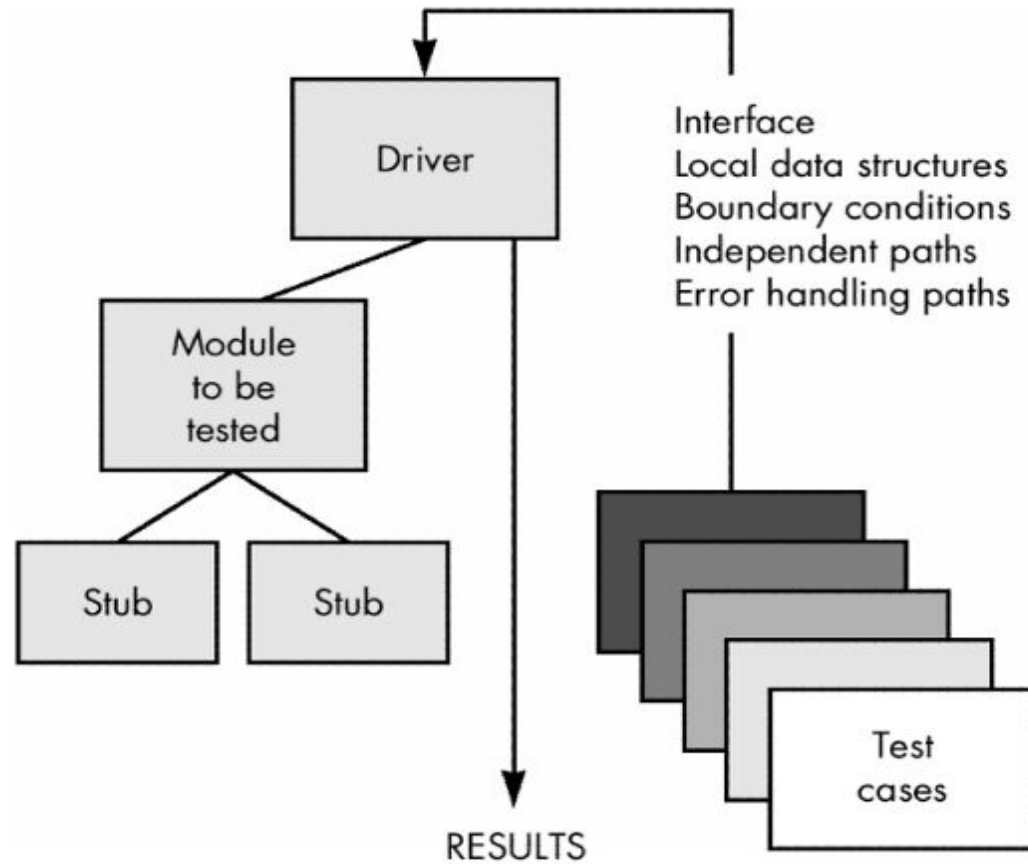
UNIT TESTING - Importance

- Once the system is integrated and you found error in an integrated system it becomes difficult to differentiate that the error occurred in which unit so unit testing is mandatory before integrating the units.
- When developer is coding the software it may happen that the dependent modules are not completed for testing, in such cases developers use stubs and drivers to simulate the called(stub) and caller(driver) units.
- Unit testing requires stubs and drivers, stubs simulates the called unit and driver simulates the calling unit.

UNIT TESTING – Stub and Driver

- Stubs and Drivers are two different types of dummy modules which are mostly used in the case of component testing.
- The main purpose of a stub is to allow testing of the upper levels of the code when the lower levels of the code are not yet developed.
- Whereas driver allows testing of the lower levels of the code, when the upper levels of the code are not yet developed.

UNIT TESTING – Stub and Driver



UNIT TESTING – Stub and Driver

Example for Stub:

Let us assume that we have 3 modules Sign-in, Log-in, and user-database module. Suppose Log-in module is ready and need to test it, but we call functions from Sign-in and user-database (which is not ready). To test at a selective module we write a short dummy piece of a code which simulates Sign-in and user-database, which will return values for Log-in, this piece of pseudo code is called Stubs and it is used in a top-down strategy.

Example for Driver:

Consider our Sign-in and user-database modules are ready and Log-in module is not ready, and we need to test Sign-in and user-database modules which return values from Log-in module, So to take out the values from Log-in module We write a Short Piece of Dummy code for Log-in which returns value for Sign-in and user-database, So these pieces of code is always called Drivers and it is used to Bottom-Up strategy.

UNIT TESTING – Stub and Driver

```
graph TD; A([Log in module]); B([Sign in module]); C([User database module]);
```

Log in
module

Sign in
module

User
database
module

UNIT TESTING – Stub and Driver

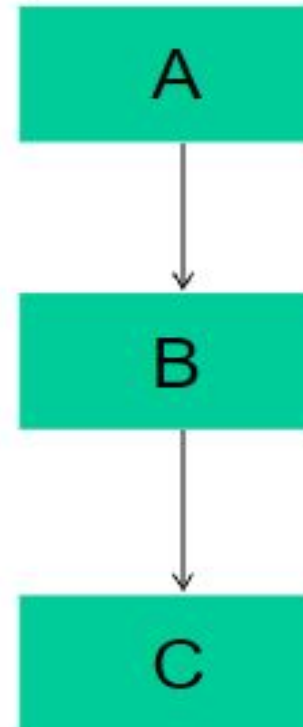
Stub

- **dummy module** which simulates the functionality of a module called by a given module under test

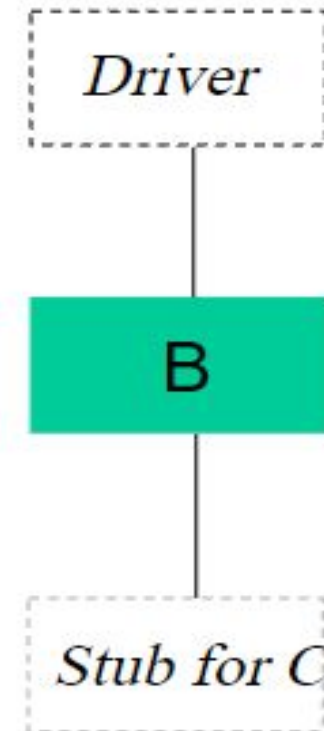
Driver

- a module which transmits test cases in the form of input arguments to the given module under test and either prints or interprets the results produced by it

e.g. module
call hierarchy



e.g. to test B
in isolation



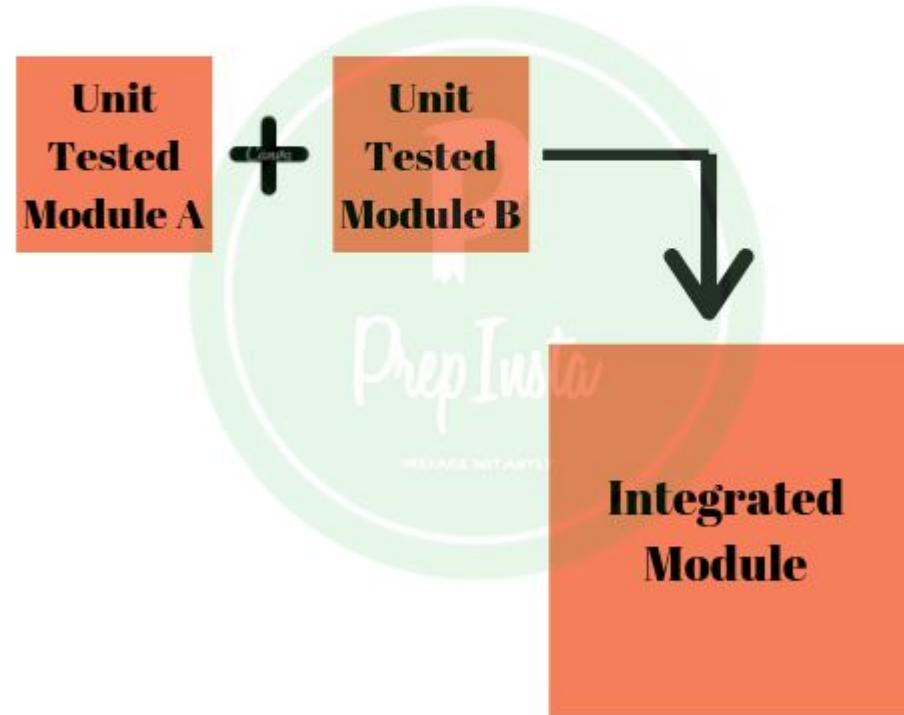
INTEGRATION TESTING

- Integration testing is a level of software testing in which units/components are integrated in a single module and then test to check communication between these components.
- The test cases of integration testing differs from other test cases i.e. it only focuses on the interfaces & flow of the data between the modules.
- This testing comes under both black box and white box testing. This testing is done by software testers or developers.

INTEGRATION TESTING



Integration Testing



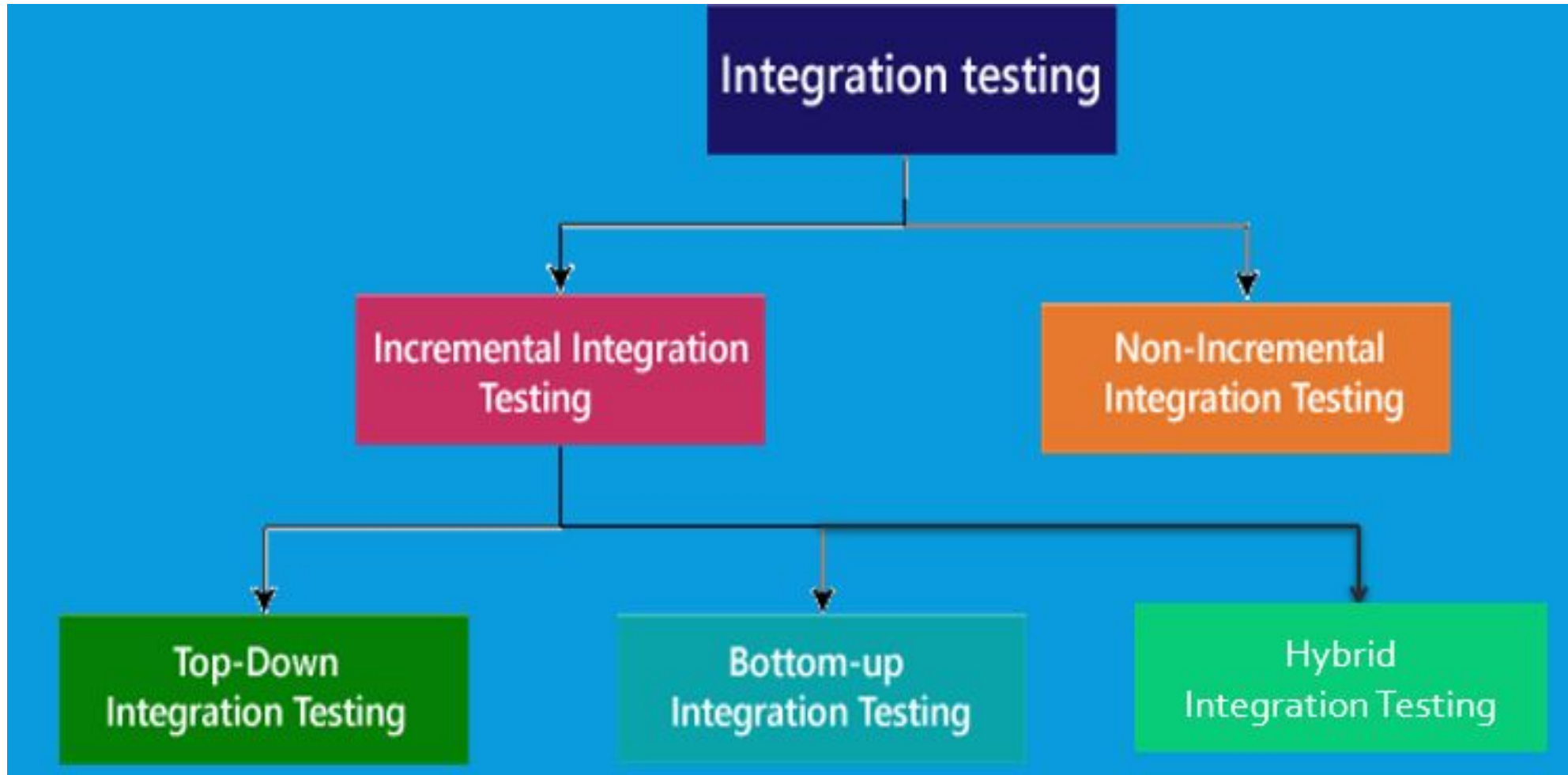
INTEGRATION TESTING- Objectives

- To reduce risk in system testing
- To verify the software modules work correctly.
- If developer deploys the change in requirements and not perform unit testing on them then integration testing is necessary.
- To build confidence in the quality of the interfaces.
- To find issues/bugs if present in the interface or components or systems.

INTEGRATION TESTING- Steps

- Prepare the test integration plan
- Decide on the type of integration testing approach
- Design test cases, test scenarios and test scripts accordingly
- Deploy the chosen modules together and get the integration tests running
- Track the defects and record the test results of tests
- Repeat the above steps until the complete system is tested

INTEGRATION TESTING- Types



INTEGRATION TESTING- 1. Incremental Approach

- In this, the modules are added in ascending order one by one or according to need. The selected modules must be logically related.
- Generally, two or more than two modules are added and tested to determine the correctness of functions.
- The process continues until the successful testing of all the modules.
- In this type of testing, there is a strong relationship between the dependent modules.
- Suppose we take two or more modules and verify that the data flow between them is working fine. If it is, then add more modules and test again.

INTEGRATION TESTING- 1. Incremental Approach

For example:

Suppose we have a Flipkart application, we will perform incremental integration testing, and the flow of the application would like this:

Flipkart → Login → Home → Search → Add cart → Payment → Logout

INTEGRATION TESTING- 1. Incremental Approach

- **Stubs and Drivers** are the dummy programs in Integration testing used to facilitate the software testing activity.
- These programs act as a substitutes for the missing models in the testing. They do not implement the entire programming logic of the software module but they simulate data communication with the calling module while testing.
- **Stub:** Is called by the Module under Test.
- **Driver:** Calls the Module to be tested.

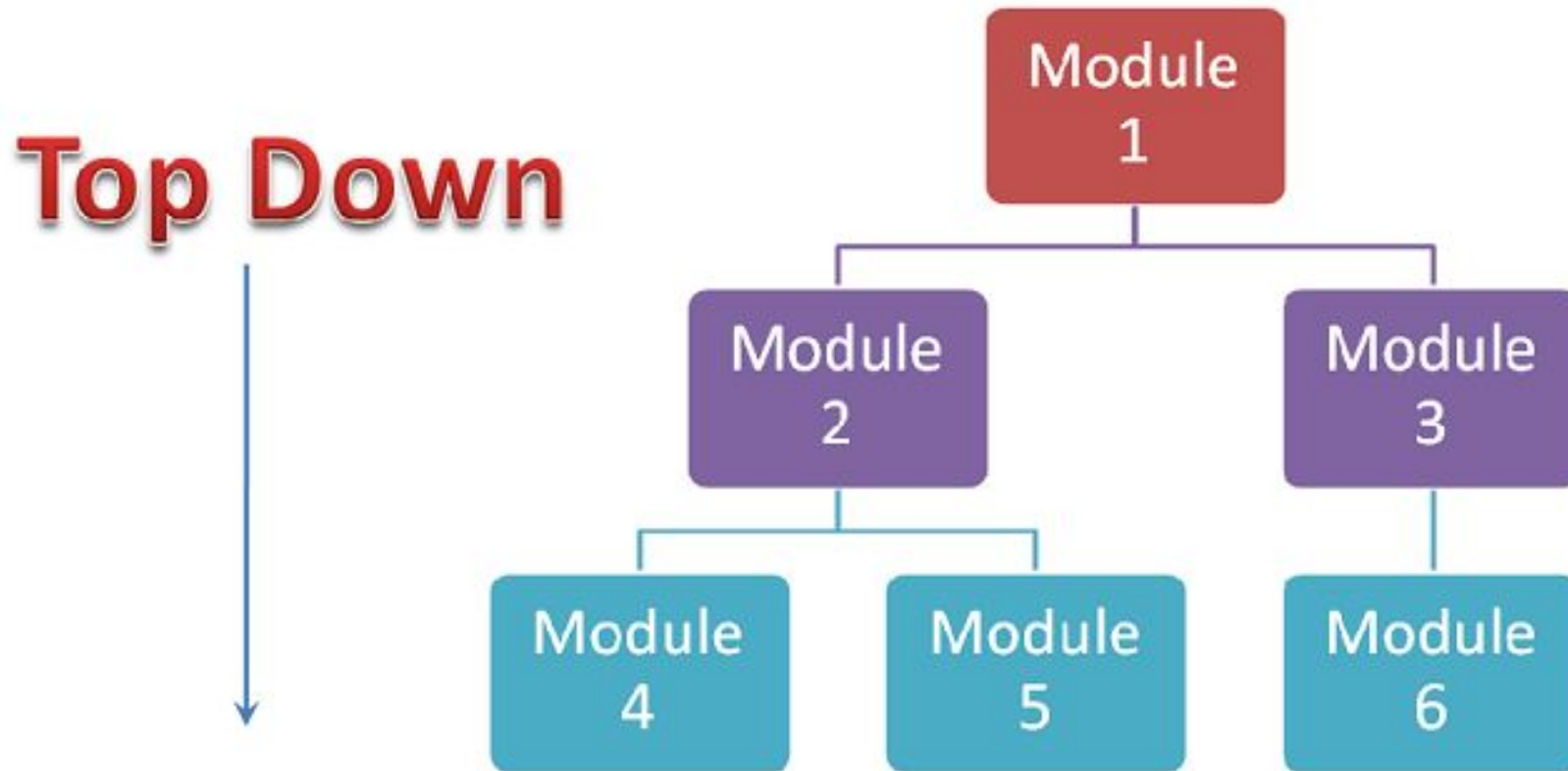
INTEGRATION TESTING- 1. Incremental Approach

1. Top-Down Integration Testing:

- It is a method in which integration testing takes place from top to bottom following the control flow of software system.
- The higher level modules are tested first and then lower level modules are tested and integrated in order to check the software functionality.
- Stubs are used for testing if some modules are not ready.

INTEGRATION TESTING- 1. Incremental Approach

1. Top-Down Integration Testing:



INTEGRATION TESTING- 1. Incremental Approach

1. Top-Down Integration Testing:

Advantages:

- Fault Localization is easier.
- Possibility to obtain an early prototype.
- Critical Modules are tested on priority; major design flaws could be found and fixed first.

Disadvantages:

- Needs many Stubs.
- Modules at a lower level are tested inadequately

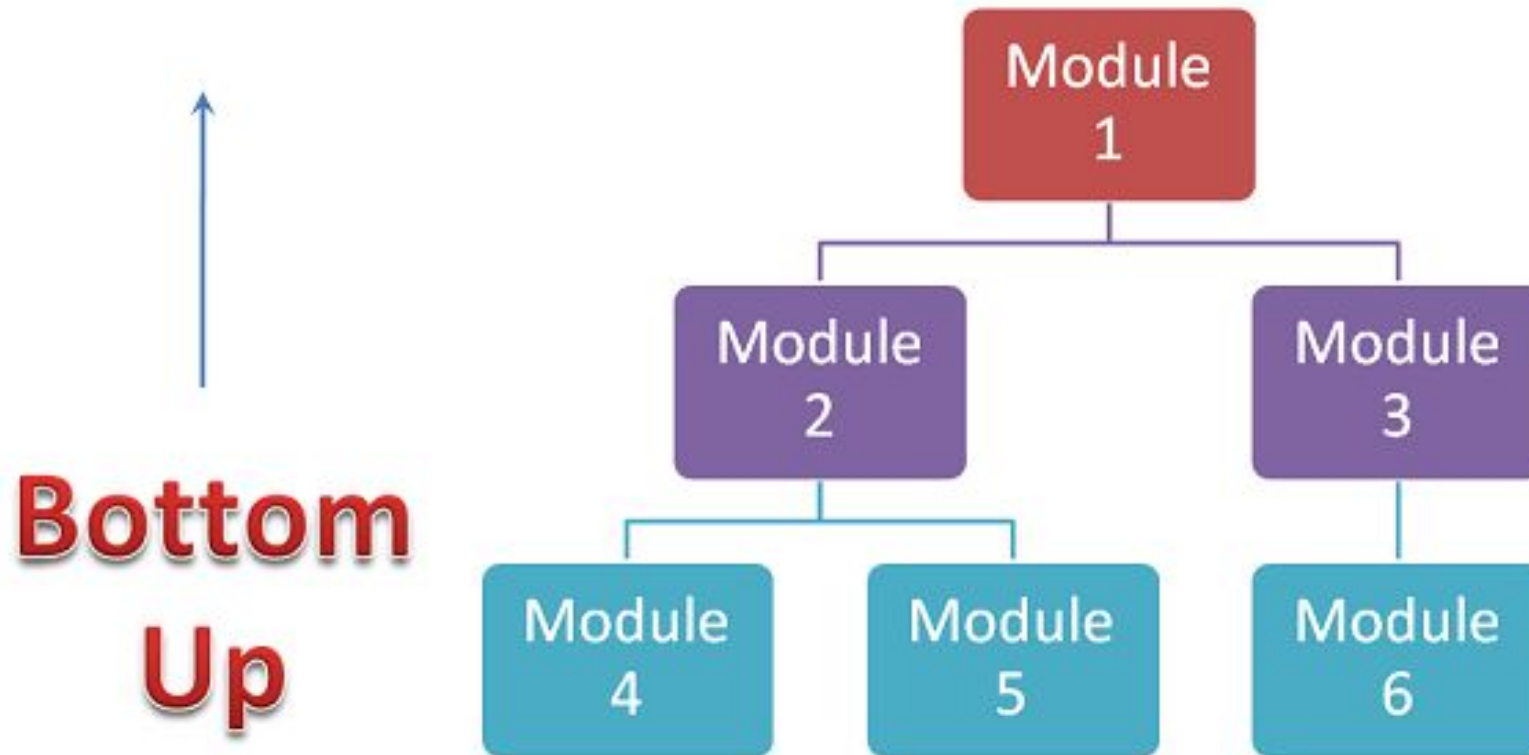
INTEGRATION TESTING- 1. Incremental Approach

2. Bottom-Up Integration Testing:

- It is a strategy in which the lower level modules are tested first.
- These tested modules are then further used to facilitate the testing of higher level modules.
- The process continues until all modules at top level are tested.
- Once the lower level modules are tested and integrated, then the next level of modules are formed.

INTEGRATION TESTING- 1. Incremental Approach

2. Bottom-Up Integration Testing:



INTEGRATION TESTING- 1. Incremental Approach

2. Bottom-Up Integration Testing:

Advantages:

- Fault localization is easier.
- No time is wasted waiting for all modules to be developed unlike Big-bang approach

Disadvantages:

- Critical modules (at the top level of software architecture) which control the flow of application are tested last and may be prone to defects.
- An early prototype is not possible

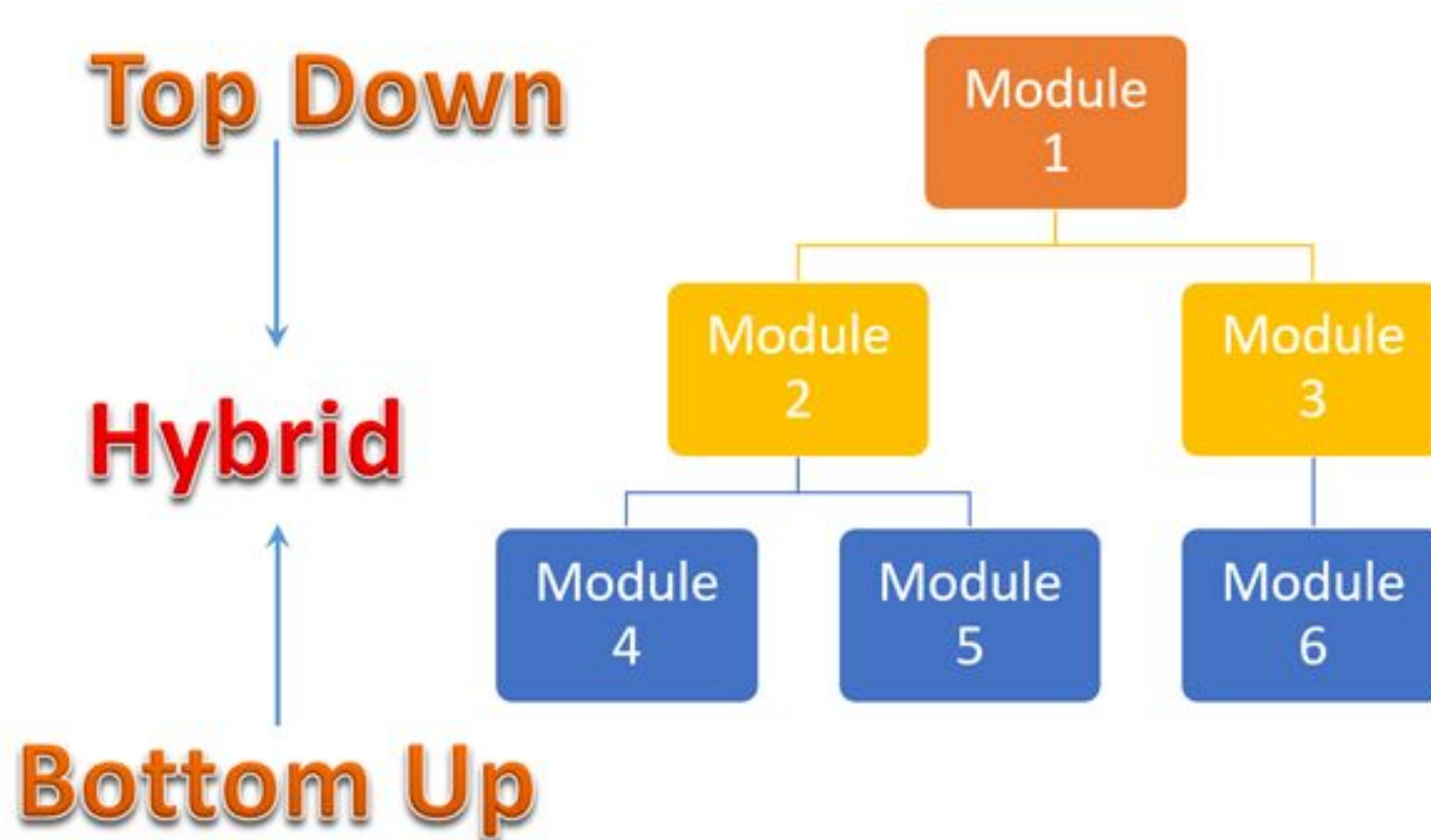
INTEGRATION TESTING- 1. Incremental Approach

3. Hybrid Integration Testing:

- It is a strategy in which top level modules are tested with lower level modules at the same time lower modules are integrated with top modules and tested as a system.
- It is a combination of Top-down and Bottom-up approaches therefore it is called **Hybrid Integration Testing**.
- It makes use of both stubs as well as drivers.

INTEGRATION TESTING- 1. Incremental Approach

3. Hybrid Integration Testing:



INTEGRATION TESTING- 2. Non-Incremental Approach

- We will go for this method, when the data flow is very complex and when it is difficult to find who is a parent and who is a child.
- And in such case, we will create the data in any module bang on all other existing modules and check if the data is present.
- Hence, it is also known as the **Big bang method**.

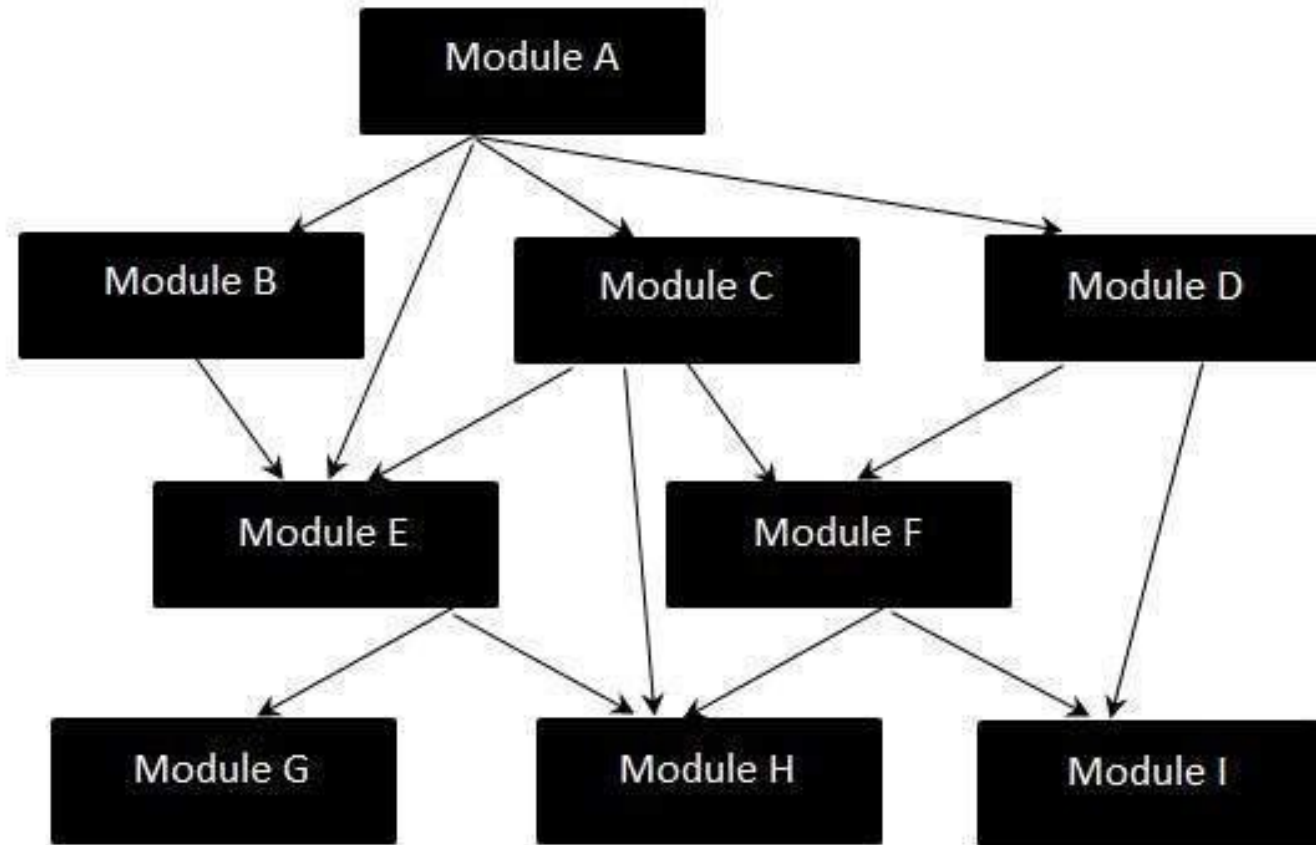
INTEGRATION TESTING- 2. Non-Incremental Approach

Big Bang Testing

- **Big Bang Testing** is an Integration testing approach in which all the components or modules are integrated together at once and then tested as a unit.
- This combined set of components is considered as an entity while testing.
- If all of the components in the unit are not completed, the integration process will not execute.

INTEGRATION TESTING- 2. Non-Incremental Approach

Big Bang Testing



INTEGRATION TESTING- 2. Non-Incremental Approach

Big Bang Testing

Advantages:

- It is convenient for small size software systems.

Disadvantages:

- Identification of defects is difficult because finding the error where it came from is a problem, and we don't know the source of the bug.
- Small modules missed easily.
- Time provided for testing is very less.
- We may miss to test some of the interfaces.

INTEGRATION TESTING- Advantages

- This testing provides faster development and increase confidence level.
- It is easy to combine different modules.
- Code coverage is high than other approaches.
- It easily caught system level issues like mistakes in integration, broken database etc.
- This testing can start when the relevant modules/components are available. It is not necessary to wait until all the modules are implemented and unit tested.
- Integration testing can be used in the early as well as later stages of the testing process and defects find easily.
- This testing makes sure that the combined modules works properly.

INTEGRATION TESTING- Disadvantages

- To be precise the success of Integration Testing lies in the perfection of the test plan.
- Test plan is made by humans one needs to serve for its inefficiencies.
- If any scenario is not covered in the test plan then errors can be occurs in the system testing.
- This testing can be time-consuming.

INTEGRATION TESTING- Example

let us consider the Yahoo.com page.



INTEGRATION TESTING- Example

- In Top Down Integration, the order of integration could be:

1. Yahoo Homepage with Mail
2. Yahoo Homepage with News
3. Yahoo Homepage with Shopping
4. Yahoo Homepage with Finance
5. Mail with Inbox
6. Mail with Sent
7. Mail with Contacts

- If News module is not fully coded, it is replaced by a skeletal implementation of the component called a STUB. For example, a static page with this a message this Page is still under construction. A Stub is called by other components.

INTEGRATION TESTING- Example

- In Bottom-Up Integration, the order of Integration could be:

1. Contacts with Mail
2. Sent with Mail
3. Inbox with Mail
4. Mail with Yahoo Homepage
5. News with Yahoo Homepage
6. Finance with Yahoo Homepage
7. Shopping with Yahoo Homepage

- Opposite of top-down, the components here are integrated in a bottom-up order. If the 'Yahoo Homepage' is yet to be fully built, special components called 'DRIVERS' are used to test the integration. The Drivers are later replaced by the full code. As we can see, Drivers are more complicated than stubs.

INTEGRATION TESTING- Example

- If the modules are all integrated together in one shot, it is called Big Bang.
- Big Bang is not preferred because it is difficult to find out which module interfaces caused the errors.
- In Big Bang, all components are put together at the same time, there is no order, except all are integrated at the same time.

SYSTEM TESTING

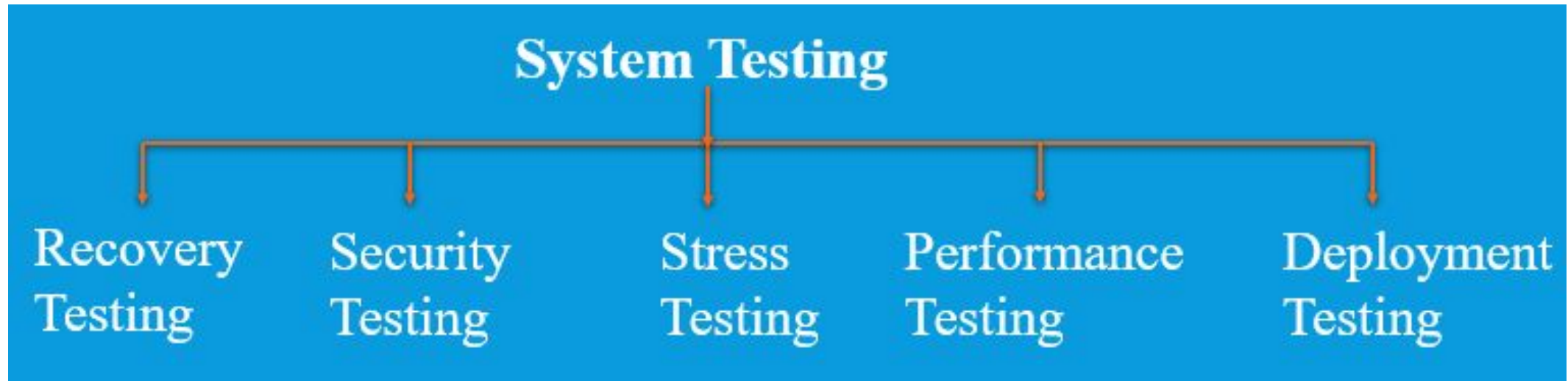
- It is a level of testing that validates the complete and fully integrated software product.
- **Purpose:** to evaluate the end-to-end system specifications.
- **System testing falls under Black box testing** as it includes testing of the external working of the software. Testing follows user's perspective to identify minor defects.

SYSTEM TESTING - Steps

- Verification of input functions of the application to test whether it is producing the expected output or not.
- Testing of integrated software by including external peripherals to check the interaction of various components with each other.
- Testing of the whole system for End to End testing.
- Behavior testing of the application via user's experience

SYSTEM TESTING - Types

System testing is divided into more than 50 types, but software testing companies typically use some of them. These are listed below:



SYSTEM TESTING - Types

1. **Recovery testing:** Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed.
2. **Security testing:** Security testing attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration.

SYSTEM TESTING - Types

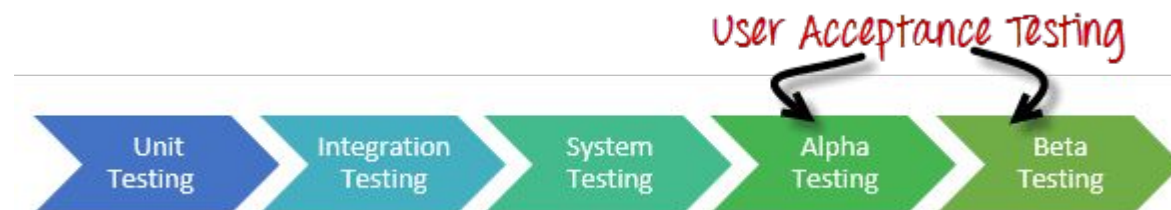
3. **Stress testing:** Stress testing executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. A variation of stress testing is a technique called sensitivity testing.
4. **Performance testing:** It is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process.

SYSTEM TESTING - Types

5. **Deployment testing:** It is also called as configuration testing. It examines all installation procedures and specialized installation software that will be used by customers, and all documentation that will be used to introduce the software to end users.

ACCEPTANCE TESTING

- **User Acceptance Testing (UAT)** is a type of testing performed by the end user or the client to verify/accept the software system before moving the software application to the production environment.
- UAT is done in the final phase of testing after functional, integration and system testing is done.
- **Types** : Alpha testing and Beta testing



ALPHA TESTING

- **Alpha Testing** is a type of acceptance testing; performed to identify all possible issues and bugs before releasing the final product to the end users.
- Alpha testing is carried out by the testers who are internal employees of the organization.
- The main goal is to identify the tasks that a typical user might perform and test them.

ALPHA TESTING

- It is done early on, near the end of the development of the software, and before beta testing.
- The main focus of alpha testing is to simulate real users by using a black box and white box techniques.

BETA TESTING

- **Beta Testing** is performed by “real users” of the software application in “real environment” and it can be considered as a form of external User Acceptance Testing.
- It is the final test before shipping a product to the customers. Direct feedback from customers is a major advantage of Beta Testing. This testing helps to test products in customer’s environment.

DIFFERENCE: ALPHA AND BETA TESTING

• Alpha testing performed by Testers who are usually internal employees of the organization	• Beta testing is performed by Clients or End Users who are not employees of the organization
• It involves both the white box and black box techniques	• It typically uses Black Box Testing

TESTING TECHNIQUES

There are two testing techniques:

- White box testing
- Black box testing

WHITE BOX TESTING

- It is a software testing technique in which internal structure, design and coding of software are tested to verify flow of input-output and to improve design, usability and security.
- In this, the code is visible to testers so it is also **called Clear box testing, Open box testing, Transparent box testing and Glass box testing.**
- It is also called as **structural testing** because it tests the internal structure of the software.

WHITE BOX TESTING

Using white-box testing methods, you can derive test cases that

1. guarantee that all independent paths within a module have been exercised at least once,
2. exercise all logical decisions on their true and false sides,
3. execute all loops at their boundaries and within their operational bounds, and
4. exercise internal data structures to ensure their validity.

WHITE BOX TESTING - Methods

- **Statement Coverage** - This technique is aimed at exercising all programming statements with minimal tests.
- **Branch Coverage** - This technique is running a series of tests to ensure that all branches are tested at least once.
- **Path Coverage** - This technique corresponds to testing all possible paths which means that each statement and branch is covered.

WHITE BOX TESTING - Methods

Statement Coverage - This technique is aimed at exercising all programming statements with minimal tests.

Example:

Read A

Read B

if A>B

Print "A is greater than B"

else

Print "B is greater than A"

Endif

Set1 :If A =5, B =2

- No of statements Executed: 5
- Total no of statements in the source code: 7
- Statement coverage = $5/7 * 100 = 71 \%$

Set2 :If A =2, B =5 [False-Not supported by Statement coverage]

- No of statements Executed: 6
- Total no of statements in the source code: 7
- Statement coverage = $6/7 * 100 = 85.20 \%$

WHITE BOX TESTING - Steps

1. Understand the source code
2. Create test cases
3. Execute

WHITE BOX TESTING - Advantages

1. White box testing is very thorough as the entire code and structures are tested.
2. It results in the optimization of code removing error and helps in removing extra lines of code.
3. It can start at an earlier stage as it doesn't require any interface as in case of black box testing.
4. Easy to automate

WHITE BOX TESTING – Disadvantages

1. Main disadvantage is that it is very expensive.
2. Redesign of code and rewriting code needs test cases to be written again.
3. Testers are required to have in-depth knowledge of the code and programming language as opposed to black box testing.
4. Very complex process

BASIS PATH TESTING

- Basis path testing is a white-box testing technique first proposed by Tom McCabe.
- In this, test cases derived to exercise the basis set are guaranteed to execute every statement in the program at least one time during testing.
- To design test cases using this technique, four steps are followed :
 1. Construct the Control Flow Graph
 2. Compute the Cyclomatic Complexity of the Graph
 3. Identify the Independent Paths
 4. Design Test cases from Independent Paths

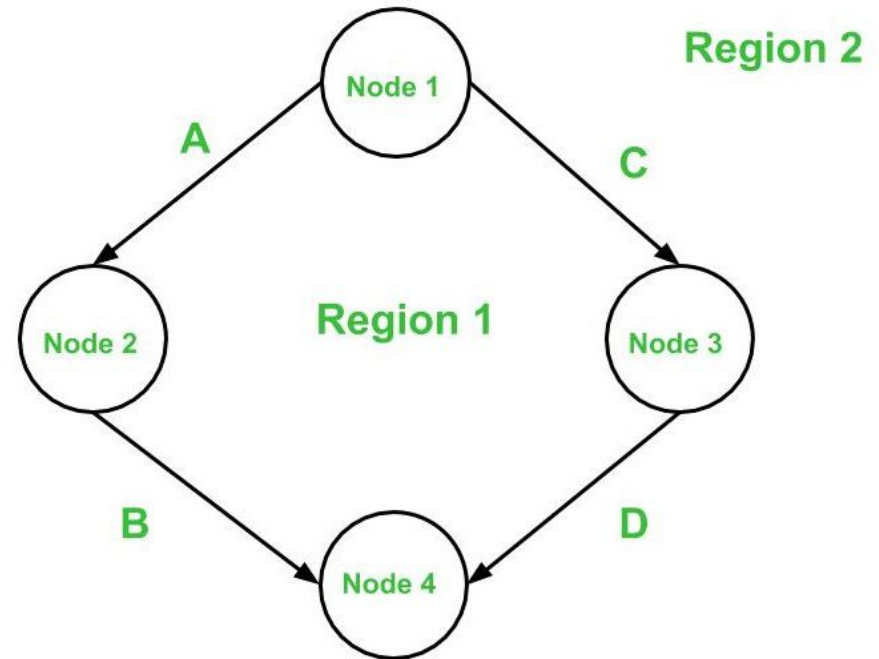
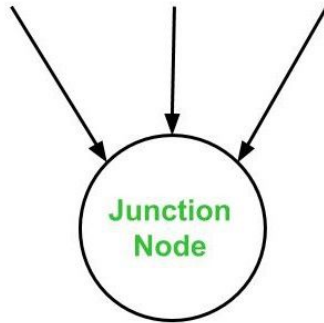
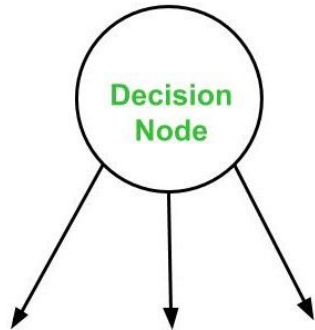
BASIS PATH TESTING

Construct the Control Flow Graph:

- It is a directed graph which represents the control structure of a program or module.
- A control graph can also have :
 1. **Junction Node** – a node with more than one arrow entering it.
 2. **Decision Node** – a node with more than one arrow leaving it.
 3. **Region** – area bounded by edges and nodes

BASIS PATH TESTING

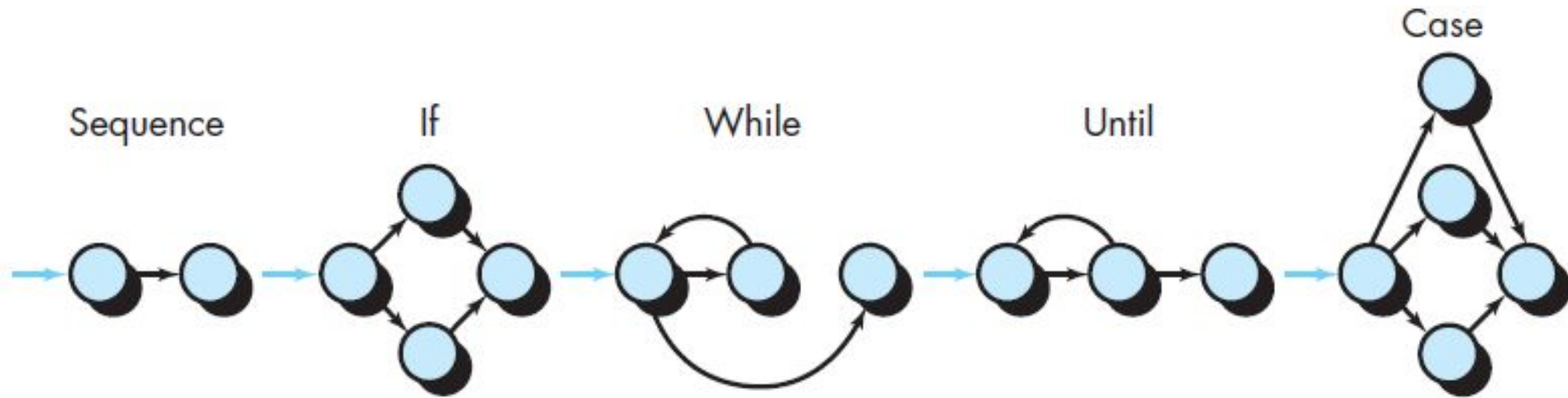
Construct the Control Flow Graph:



BASIS PATH TESTING

Construct the Control Flow Graph:

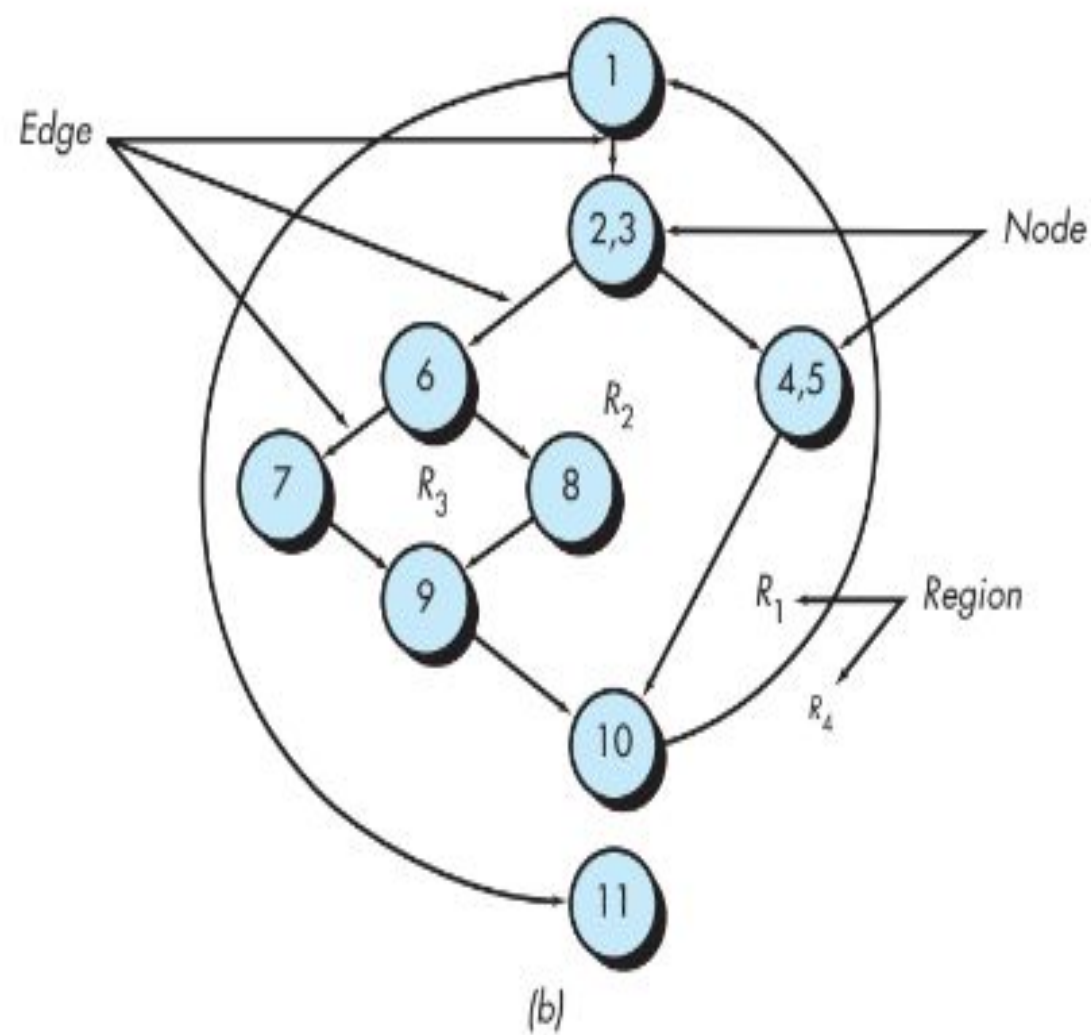
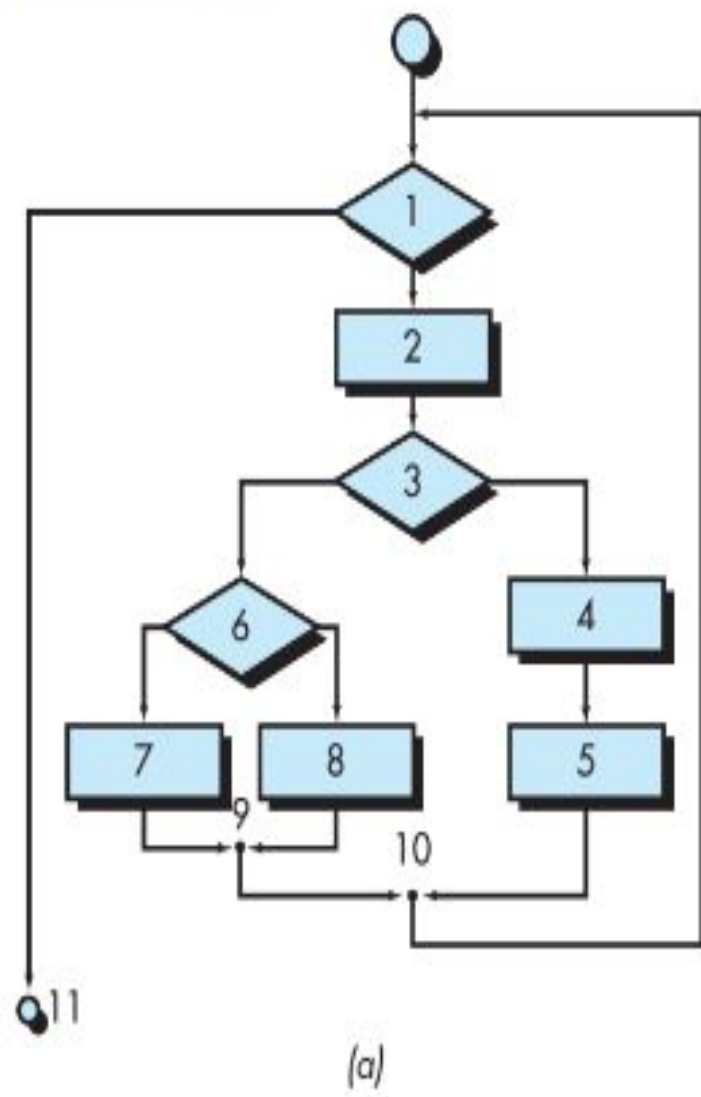
Below are the **notations** used while constructing a flow graph :



BASIS PATH TESTING

Compute the Cyclomatic Complexity of the Graph:

- Cyclomatic complexity is a software metric and another key process in implementing basis path testing.
- cyclomatic complexity measures the complexity of a program by identifying all independent paths through which the processes flow.

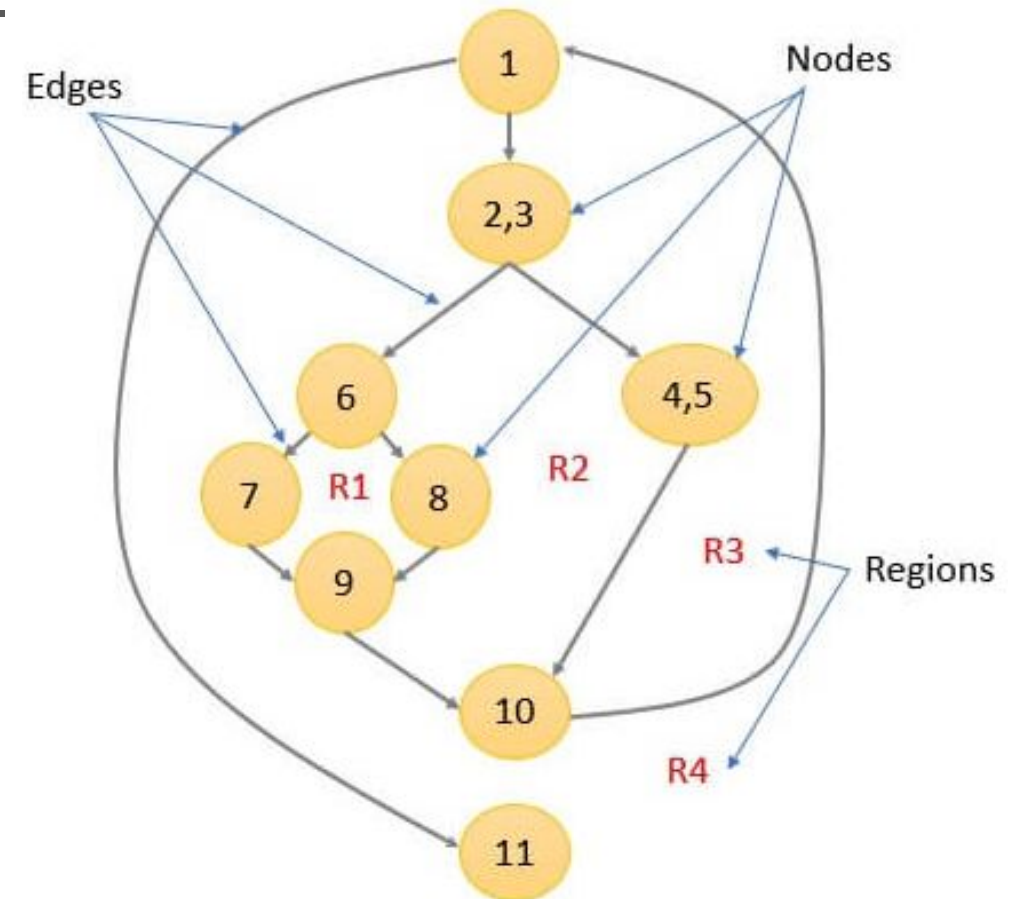


BASIS PATH TESTING

Compute the Cyclomatic Complexity of the Graph:

The cyclomatic complexity $V(G)$ can be calculated with the formulas below:

1. $V(G) = E - N + 2$ // E is edges, N is nodes
 $V(G) = 11 \text{ edges} - 9 \text{ nodes} + 2 = 4$
2. $V(G) = P + 1$ // P is predicate node
 $V(G) = 3 \text{ predicate nodes} + 1 = 4$
3. $V(G) = \text{no. of regions of a flow graph} = 4$



BASIS PATH TESTING

Identify the Independent Paths:

- An independent path is any path through the program that introduces at least one new set of processing statements or a new condition.
- When stated in terms of a flow graph, an independent path must move along at least one edge that has not been traversed before the path is defined.

BASIS PATH TESTING

Identify the Independent Paths:

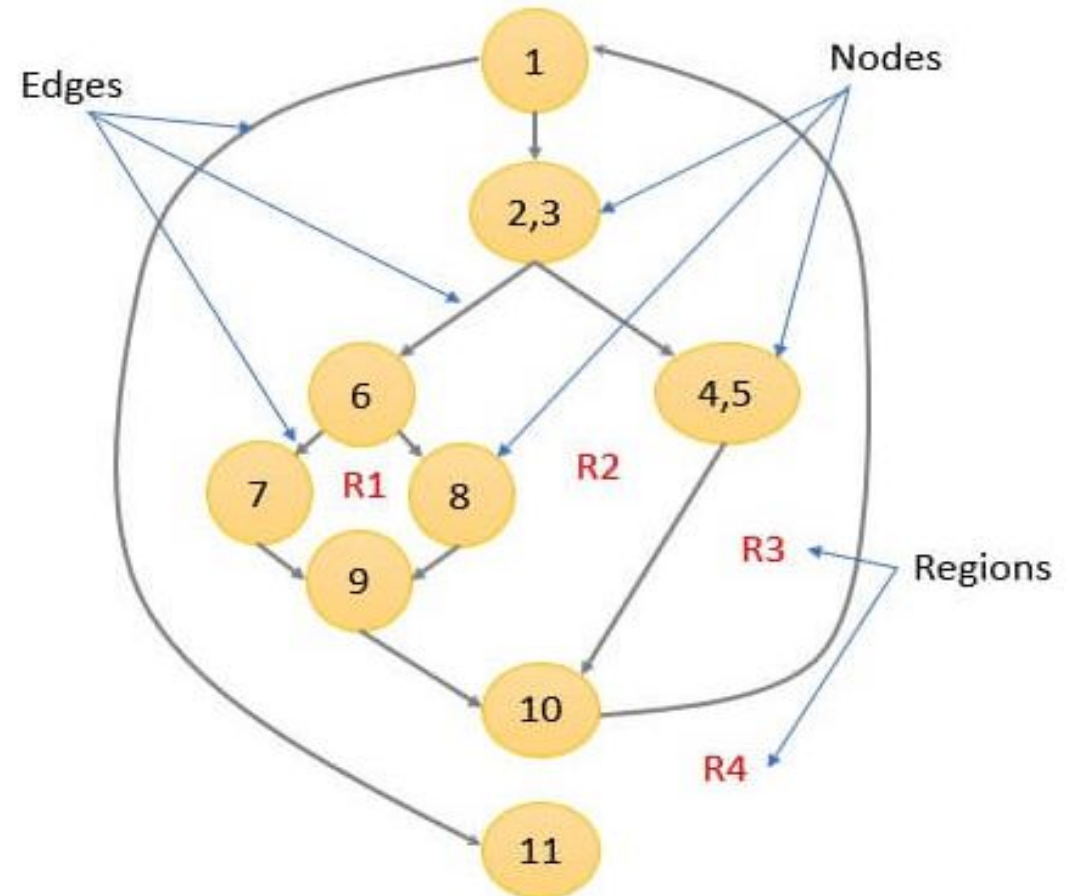
All the independent path that can be traced in the flow graph should start from 1 and end to 11:

Path 1: 1-11

Path 2: 1-2-3-4-5-10-1-11

Path 3: 1-2-3-6-8-9-10-1-11

Path 4: 1-2-3-6-7-9-10-1-11



BASIS PATH TESTING

Design Test cases from Independent Paths:

Step 1: The first step is to **draw a flow graph** by reviewing the source code of the program or its procedural design.

Step 2: Reviewing the flow graph you have to **compute cyclomatic complexity**.

Step 3: The value computed by cyclomatic complexity provides the number up to which independent paths could be derived to **determine the basis set**.

Step 4: Reviewing the basis set testers **design the test case** that will exercise each statement in the program and every side of conditional statements in the program.

BASIS PATH TESTING - Advantages

1. This provides a systematic way of designing test cases.
2. The cyclomatic complexity provides quantitative measures to derive the number of independent paths in the program. So, it becomes easy for testers to determine how many independent paths they have to look for.
3. The test case designed with this technique covers all the statements of the program and ensures that each statement is executed at least once during testing.
4. It uncovers all the possible errors of the program.

Example : 2

Step 1: Flow Graph Example

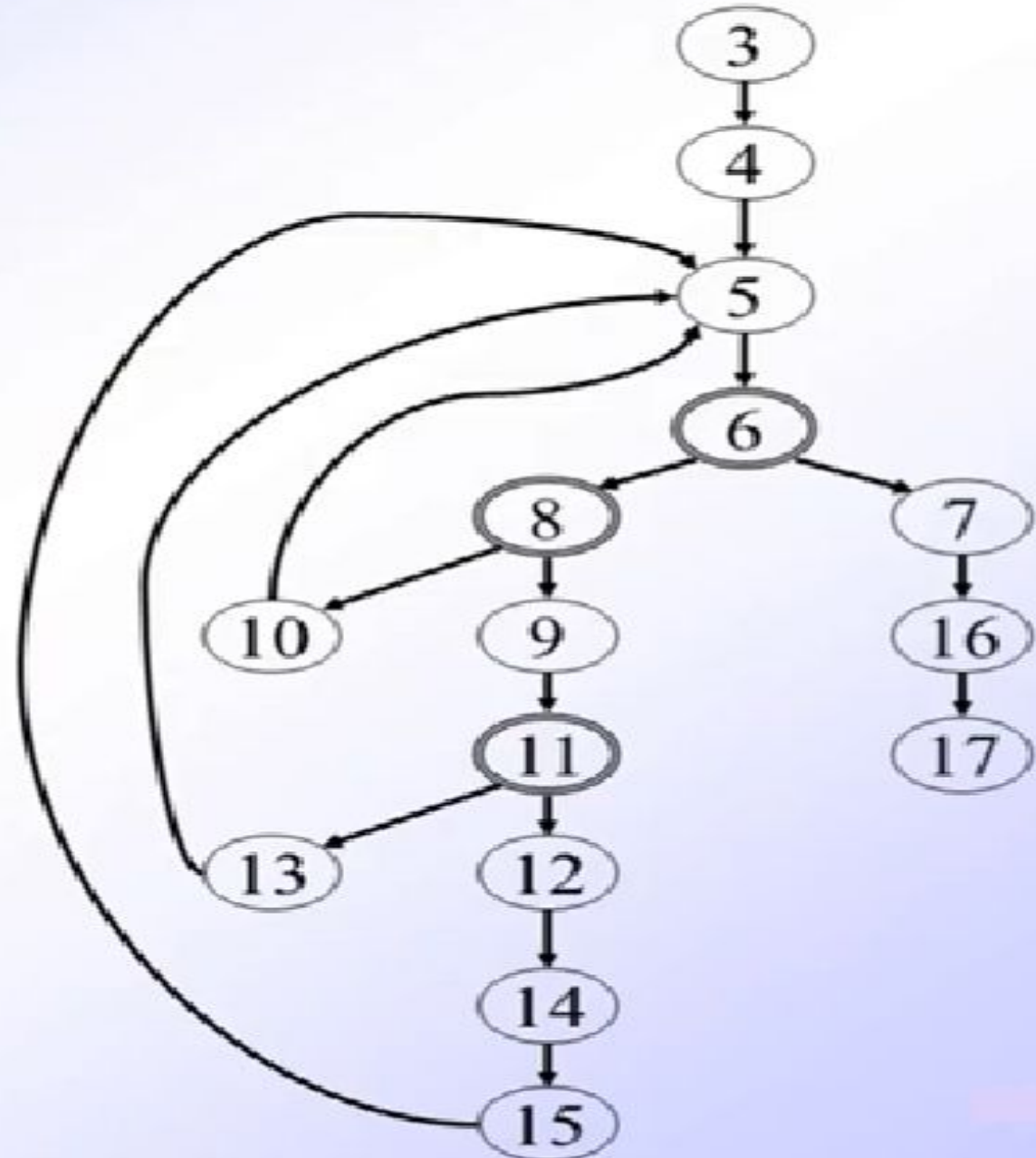
```
1  int functionY(void)
2  {
3      int x = 0;
4      int y = 19;

5  A: x++;
6      if (x > 999)
7          goto D;
8      if (x % 11 == 0)
9          goto B;
10     else goto A;

11 B: if (x % y == 0)
12     goto C;
13     else goto A;

14 C: printf("%d\n", x);
15     goto A;

16 D: printf("End of list\n");
17     return 0;
18 }
```



Control Structure Testing

Control Structure Testing is a white-box software testing technique that focuses on verifying the logical flow of a program—how different control structures (like loops, conditionals, and sequences) behave during execution.

1. Conditional Testing
2. Data Flow Testing
3. Loop Testing

Condition Testing

- Condition testing is a test case design method, which ensures that the logical condition and decision statements are free from errors. Tests individual logical conditions (e.g., if $(a > b \ \&\& \ c < d)$).
- Ensures each condition evaluates correctly (true/false).
- Includes:
 - Simple condition testing
 - Compound condition testing :When multiple conditions are combined using AND (&&) or OR (||)

Data Flow Testing

- **Data Flow Testing** is a white-box testing technique that selects test paths of a program based on:
- The points where variables are defined (assigned values)
- The points where those variables are used

Definition (def)

- A variable is **defined** when it is assigned a value.
Example: $x = 10 \rightarrow x$ is defined.

Use (use)

- A variable is **used** when its value is accessed. There are two types:
- **c-use (computational use)**: Used in calculations
Example: $y = x + 5$
- **p-use (predicate use)**: Used in conditions
Example: $\text{if } (x > 0)$

The method designs test cases to ensure:

- Every **definition of a variable** is tested with its **subsequent uses**
- No variable is used without being properly defined first

Example

```
1: x = 5      # def(x)
2: if x > 0:   # p-use(x)
3:   y = x+2  # c-use(x), def(y)
4: print(y)   # c-use(y)
```

DU Pairs:

- x: (1 $\rightarrow$ 2), (1 $\rightarrow$ 3)
- y: (3 $\rightarrow$ 4)

Test Goal:

- Ensure execution paths cover all these definition–use relationships.



Loop Testing

Loop Testing is a white-box testing technique that focuses on validating loops in a program—ensuring they execute correctly under different conditions and boundaries.

- Focuses on loops (for, while, do-while).
- Verify loops run the **correct number of times**
- Detect **infinite loops**
- Check **boundary conditions**
- Ensure correct behavior when loops are skipped

BLACK BOX TESTING

- It is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths.
- It is also known as Behavioral Testing.



BLACK BOX TESTING - Steps

- Initially, the requirements and specifications of the system are examined.
- Tester chooses valid inputs.
- Tester determines expected outputs for all those inputs.
- Software tester constructs test cases with the selected inputs.
- The test cases are executed.
- Software tester compares the actual outputs with the expected outputs.
- Defects if any are fixed and re-tested.

BLACK BOX TESTING - Types

1. Functional testing:

- Black box testing can test specific functions or features of the software under test.
- Functional testing can focus on the most critical aspects of the software, on integration between key components (integration testing), or on the system as a whole (system testing).

BLACK BOX TESTING - Types

2. Non-Functional testing:

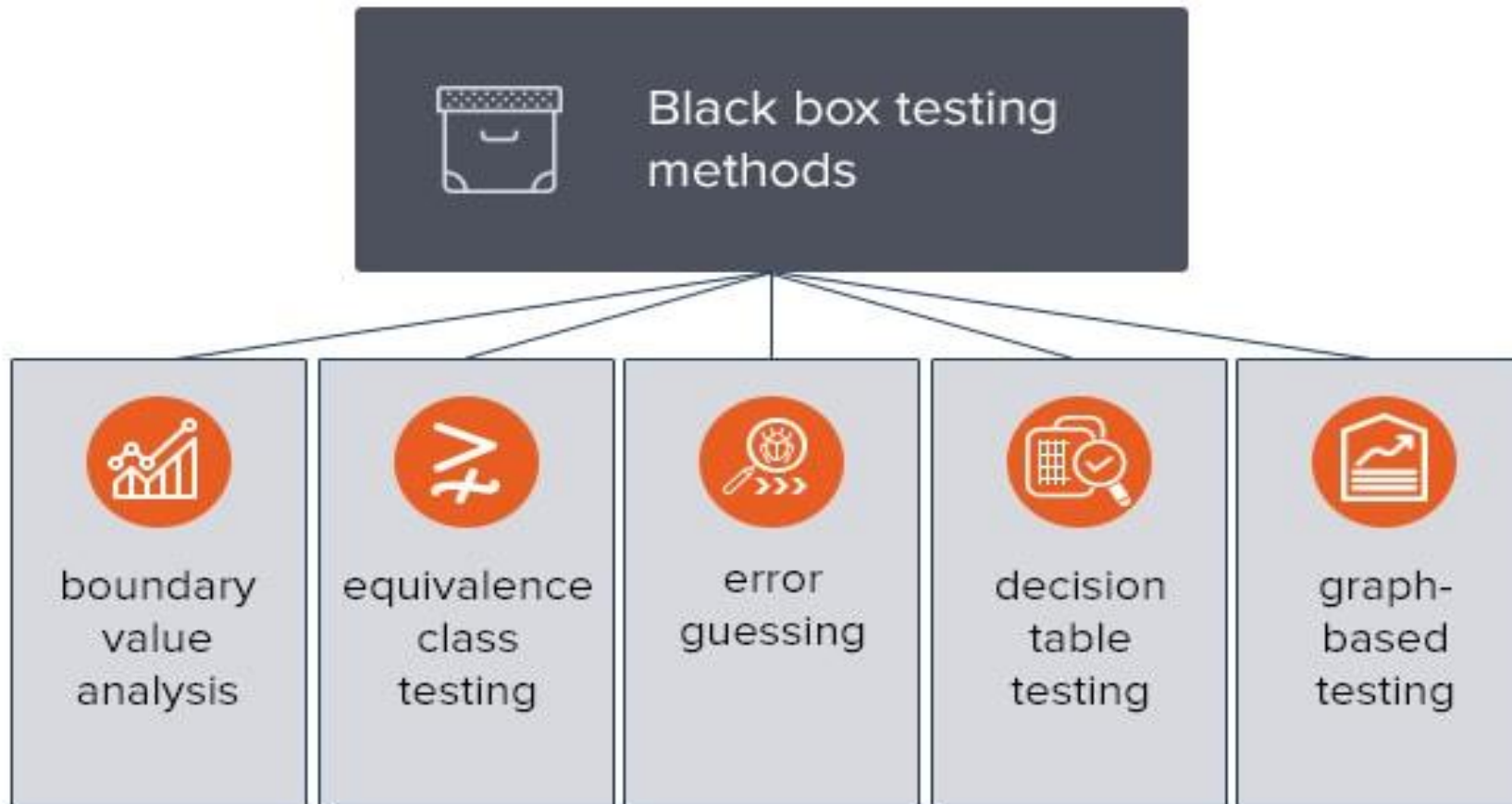
- Black box testing can check additional aspects of the software, beyond features and functionality.
- This type of black box testing is not related to testing of specific functionality, but non-functional requirements such as performance, scalability, usability.

BLACK BOX TESTING - Types

3. Regression testing:

- Regression Testing is done after code fixes, upgrades or any other system maintenance to check the new code has not affected the existing code.

BLACK BOX TESTING – Techniques/Methods



BLACK BOX TESTING - Techniques

1. Boundary Value Analysis:

- Boundary value testing is focused on the values at boundaries. This technique determines whether a certain range of values are acceptable by the system or not.
- It is very useful in reducing the number of test cases.
- It is most suitable for the systems where an input is within certain ranges.

- **Boundary Value Analysis (BVA)** is a software testing technique that focuses on testing the boundary values of input domains. Since errors often occur at the edges of input ranges, BVA targets these critical points to catch defects effectively.
- Test inputs at their minimum and maximum limits Also test values just inside and just outside the boundaries Helps detect errors related to handling of limits.

- Example: Suppose an input field accepts integers from 1 to 100.

- Boundary values to test: Test Input Description

0	Just below lower bound
1	At lower bound
2	Just above lower bound
99	Just below upper bound
100	At upper bound
101	Just above upper bound

- **Problem 1: Age Validation**

A system accepts age inputs between **18 and 60** (inclusive).

Task:

List all test inputs for Boundary Value Analysis.

- **Step 1: Identify bounds**

Lower bound = 18

Upper bound = 60

- **Step 2: List test inputs**

Just below lower bound: **17**

At lower bound: **18**

Just above lower bound: **19**

Just below upper bound: **59**

At upper bound: **60**

Just above upper bound: **61**

BLACK BOX TESTING - Techniques

2. Equivalence Class Partitioning:

- Equivalence partitioning is a technique of software testing in which input data divided into partitions of valid and invalid values, and it is mandatory that all partitions must exhibit the same behavior.
- It is used to minimize the number of possible test cases to an optimum level while maintains reasonable test coverage.
- The idea is that all values in a partition should be treated the same by the software, so testing one value from each partition is sufficient.

- **Example:**

Suppose an input accepts integers from **1 to 100**.

Class Type	Description	Example Values
Valid class	Inputs within range	1, 50, 100
Invalid class	Inputs less than 1	0, -10
Invalid class	Inputs greater than 100	101, 150

BLACK BOX TESTING - Techniques

3. Graph-based Testing:

- Each application is built by using some objects. All the objects which are used are noted and a graph is prepared.
- From this graph, the relationship of every object is identified, and test cases are written accordingly.

BLACK BOX TESTING - Techniques

4. Decision Table Testing:

- Whenever there are logical conditions or decision-making steps then this technique is to be used.
- These can be like if a particular condition is not satisfied then Action A should be performed, else Action B is to be performed.
- The tester needs to identify the input and actions which are to be performed based on the conditions. A decision table is created based on these.

BLACK BOX TESTING - Techniques

5. Error Guessing Testing:

- Error guessing is based on the previous experience of a tester i.e., the tester uses his/her previous experience about the application and functionalities to guess the error-prone areas

BLACK BOX TESTING - Advantages

1. There is no technical or programming language knowledge is required. The software tester does not need to have any knowledge of the internal structure of the system.
2. Black-box testing is more effective for large and complex applications.
3. Defects and inconsistency in the system can be identified easily so that they can be fixed.
4. The test cannot be compromised because the designer, programmer, and tester are independent.
5. Tests are carried out as soon as the programming and specifications are completed.

BLACK BOX TESTING - Disadvantages

1. The tester may ignore possible conditions of scenarios to be tested due to lack of programming and technical knowledge.
2. Complete coverage in cases of the large and complex project is not possible.
3. Testing every input stream is unrealistic because it would take a large amount of time; therefore, many program parts go untested.

DIFFERENCES BETWEEN BLACK BOX TESTING VS WHITE BOX TESTING

Black box testing	White box testing
1. Black box testing is the Software testing method which is used to test the software without knowing the internal structure of code or program.	1. White box testing is the software testing method in which internal structure is being known to tester who is going to test the software.
2. Testing is applicable on higher levels of testing like System Testing, Acceptance testing.	2. Testing is applicable on lower level of testing like Unit Testing, Integration testing.
3. Black box testing means functional test or external testing.	3. White box testing means structural test or interior testing.

DIFFERENCES BETWEEN BLACK BOX TESTING VS WHITE BOX TESTING

Black box testing	White box testing
4. Black Box testing can be started based on Requirement Specifications documents.	4. White Box testing can be started based on Detail Design documents.
5. It is performed by the end user, developer, and tester.	5. Usually it is done by tester and developers.
6. It is less exhaustive and time-consuming.	6. Exhaustive and time-consuming method.